



Vietnam National University of HCMC  
International University  
School of Computer Science and Engineering



---

# Object Oriented Programming

# Introduction

**Instructor: Le Thi Ngoc Hanh, Ph.D**

ltnhanh@hcmiu.edu.vn

Semester – Spring 2026

# Outline



- ☾ Introduction
- ☾ WHAT & WHY OOP
- ☾ Grading scheme
- ☾ Course policy
- ☾ Basics in OOP

# References

---

- ☾ [R1] ] Paul J. Deitel (Author), Harvey Deitel (Author), Java How To Program, 11th Edition, Prentice Hall, 2017
- ☾ [R2] Matt Weisfeld, The Object-Oriented Thought Process, 5th Edition, Addison-Wesley, 2019
- ☾ [R3] Erich Gamma, Richard Helm, Ralph Johnson and John Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley Professional, 1994

# About me

---



<https://lehanhcs.github.io>

# About TAs

---



## **Le Huynh Thien**

Email / MS Teams:

ITCSIU23036@student.hcmiu.edu.vn

*Support for tutorials and course Q&A in  
Group 01*



## **Nguyen Hoang Phuong**

Email / MS Teams:

ITCSIU23056@student.hcmiu.edu.vn

*Support for tutorials and course Q&A in  
Group 02*

# MS Team

---



Or join via code: **ayftat2**

# Syllabus – Course page

---



[https://lehanhcs.github.io/course\\_OOP\\_S26.html](https://lehanhcs.github.io/course_OOP_S26.html)

# Course Q&A

## ★ Communication:

Email, MS Team, office hours - by an appointment @Room A1. 610

## ★ Take attendance?



## ★ Food?



## ★ Zzz ....?



## ★ Late submissions?



## ★ Discussion is encouraged, but **plagiarism** is not allowed!

Do not use the **codes** on webpages without **modification**!

# Be Responsible for Yourself

☾ Cheating or plagiarism will result in zero score

⇒ **Final score = 0**

☾ Under 50?

⇒ **See you next year ~**



# Grading Scheme

---

The final grade for the course will be determined as follows:

C1 – Attendance/in-class quiz: 10%

C2 – Assignments: 15%

C3 – Midterm test: 30%

C4 - Final exam: 40%

C5 - Extra credit will be granted periodically for clever or creative solutions: 5%

# Evaluation Methods

---

- ☾ Midterm exam: **open** a handwritten A4 note
- ☾ Final exam: **open** a handwritten notebook
- ☾ Project: per guidance (Lab session)

# Everything Around You Is an Object

---



# From Real World to Software

## Real world



Person



## Software



Person object



Car



Car object



Bank  
Account



BankAccount  
object

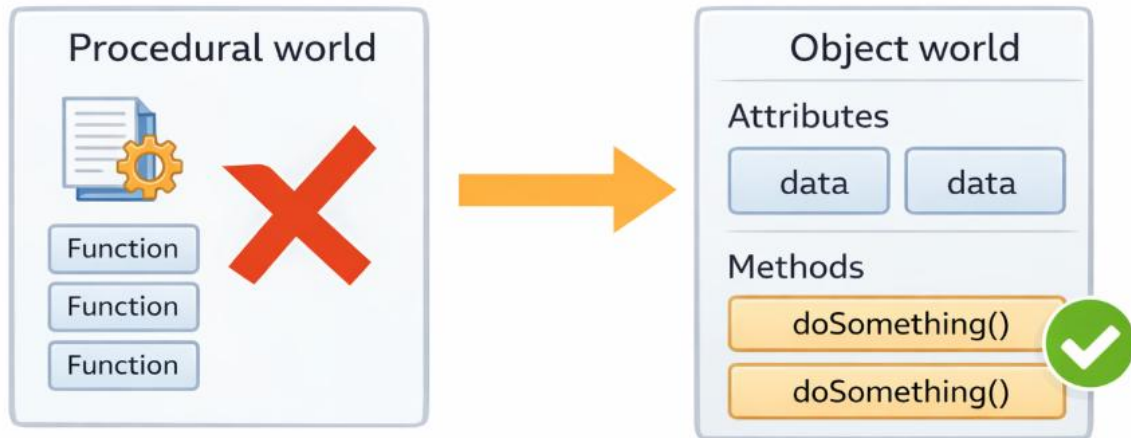
# What Is an Object?

- ★ An object is an entity that combines **data** and **behavior**.



# What Is OOP?

- OOP is a way of building programs by organizing them around **objects**, not **procedures**.



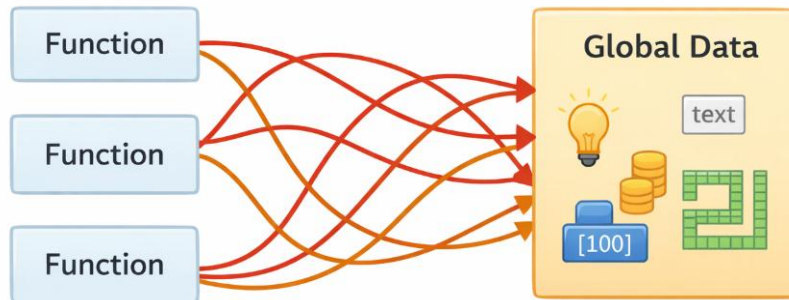


# Why OOP?

# How Programs Start?

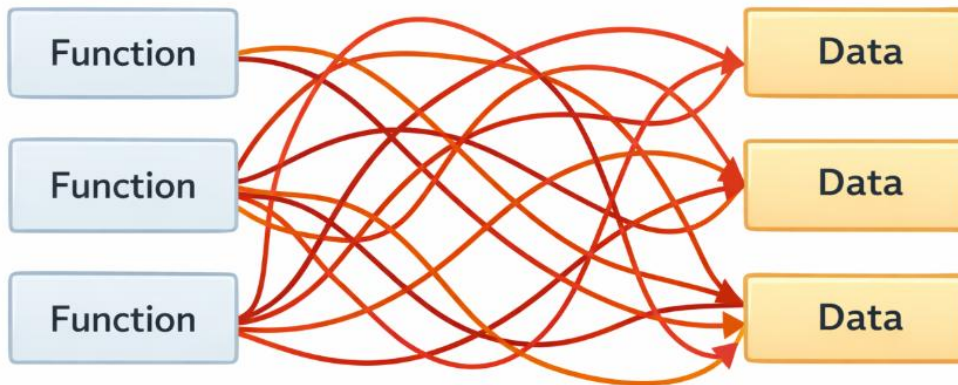
## Procedural Programming Characteristics

- ☾ Program = collection of functions
- ☾ Data is often global
- ☾ Data and functions are separated



# When Programs Grow ...

**Scenario:** 5 developers, 500 functions, 300 global variables, ....



# Core Problems

---

## **Problems with procedural style:**

- ❑ No clear ownership of data
- ❑ Anyone can modify anything
- ❑ Hard to maintain
- ❑ Hard to scale

# OOP as a Solution

## OOP solves this by:

- Giving data an owner (object)
- Controlling access
- Grouping responsibility



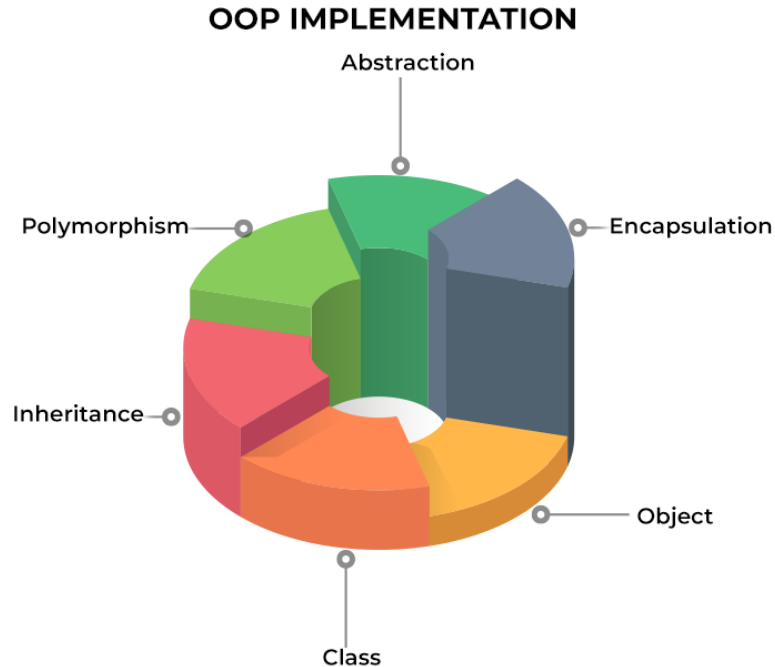
“In OOP, data is no longer homeless”

# Course Objective



- ✧ Understand limitations of procedural programming
- ✧ Explain what objects and classes are
- ✧ Understand encapsulation and data hiding
- ✧ Recognize inheritance, polymorphism, and composition
- ✧ Read basic UML class diagrams

# Basic OOP Concepts



# Class vs Object

- ★ Class = blueprint
- ★ Object = real house

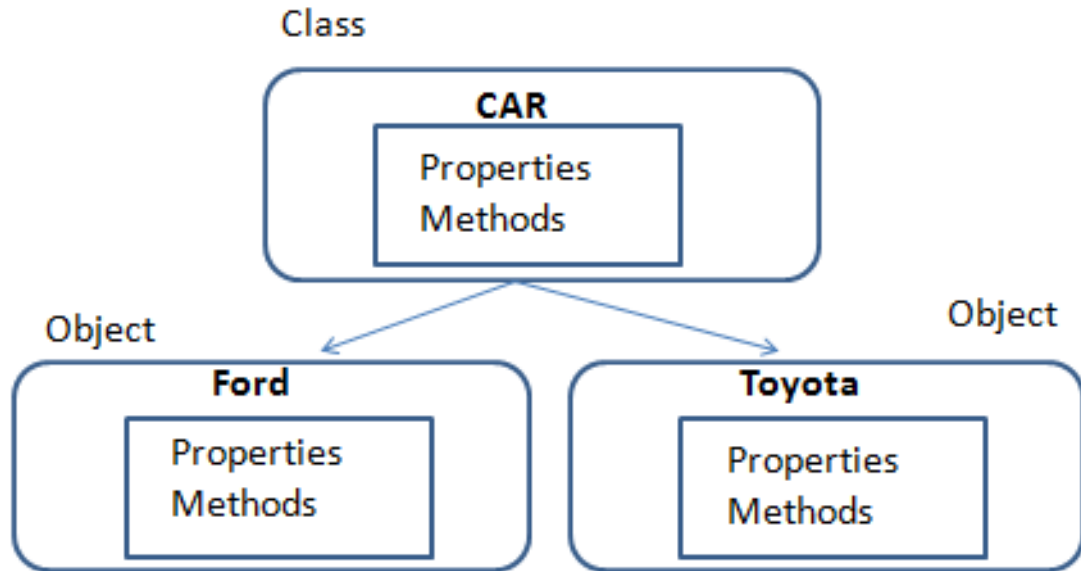


Class



Objects

# Class vs Object



# Data Hiding

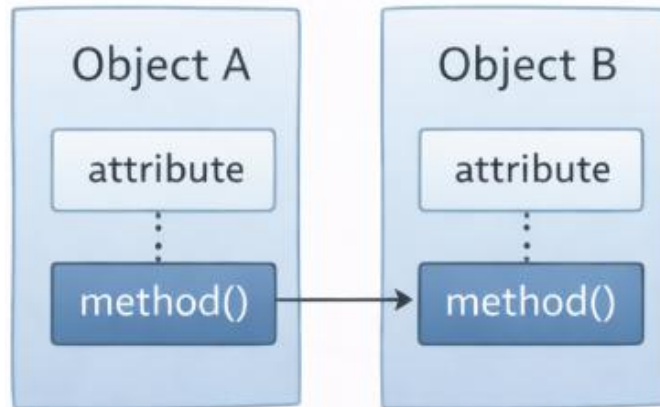
- Attributes should be private
- Access via methods



# Object Communication

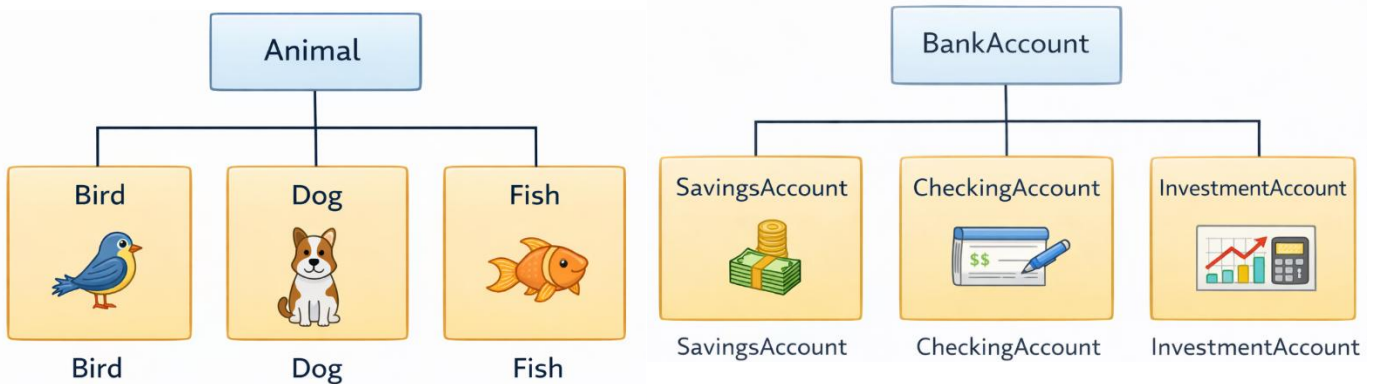
## How objects interact:

- Objects send messages
- Message = method call



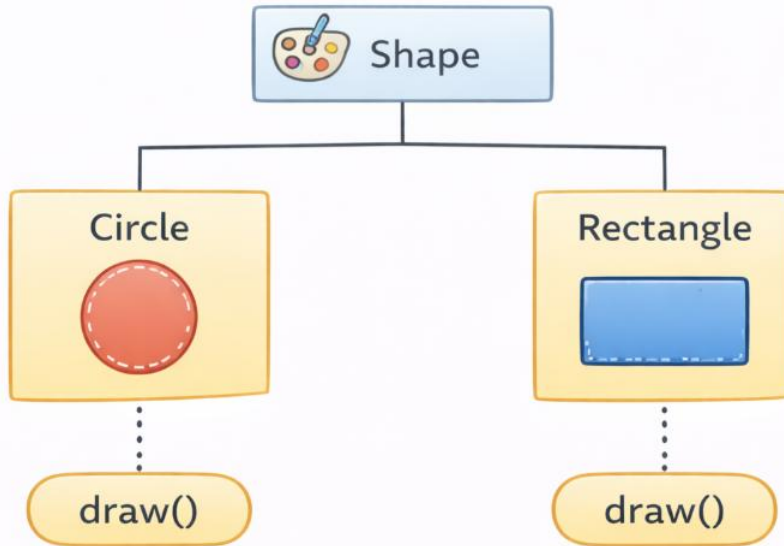
# Inheritance (Is-a)

**Concept:** Reuse code & Specialization



# Polymorphism

Polymorphism lets the same message trigger different behavior depending on the object



# Composition (Has-a)

- ☾ Objects contain other objects

